```
from config import *
from state import State
import math
import random
import heapq
_enemy_shadow = pygame.Surface((30, 10), pygame.SRCALPHA)
pygame.draw.ellipse(_enemy_shadow, (0, 0, 0, 45), (0, 0, 30, 10))
_SQRT2 = math.sqrt(2)
def _heuristic(ax, ay, bx, by):
    dx = abs(bx - ax)
    dy = abs(by - ay)
    return max(dx, dy) + (_SQRT2 - 1) * min(dx, dy)
_walk_grid_cache = {}
_walk_grid_hash = 0
def _build_walk_grid(walls):
    global _walk_grid_hash
    if not walls:
        return [], 0, 0
    h = hash((len(walls), *(w.x for w in walls), *(w.y for w in walls)))
    if h == _walk_grid_hash and _walk_grid_cache:
        return _walk_grid_cache["grid"], _walk_grid_cache["cols"], _walk_grid_cache["rows"]
    map_cols = max(w.x for w in walls) // TILE_SIZE + 1
    map_rows = max(w.y for w in walls) // TILE_SIZE + 1
    grid = [[True for _ in range(map_cols)] for _ in range(map_rows)]
    for wall in walls:
        col = wall.x // TILE_SIZE
        row = wall.y // TILE_SIZE
        if 0 <= col < map_cols and 0 <= row < map_rows:
            grid[row][col] = False
    _walk_grid_hash = h
    _walk_grid_cache["grid"] = grid
    _walk_grid_cache["cols"] = map_cols
    _walk_grid_cache["rows"] = map_rows
    return grid, map_cols, map_rows
def _astar(start, goal, grid):
    sx, sy = int(start[0]), int(start[1])
    gx, gy = int(goal[0]), int(goal[1])
    rows = len(grid)
    cols = len(grid[0])
    if not (0 <= sx < cols and 0 <= sy < rows and grid[sy][sx]):
        return []
    if not (0 <= gx < cols and 0 <= gy < rows and grid[gy][gx]):
        # Try to find nearest walkable goal
        best = None
        best_dist = float("inf")
        for dr in range(-2, 3):
            for dc in range(-2, 3):
                nr, nc = gy + dr, gx + dc
                if 0 <= nc < cols and 0 <= nr < rows and grid[nr][nc]:
                    d = abs(dr) + abs(dc)
                    if d < best_dist:
                        best_dist = d
                        best = (nc, nr)
        if best is None:
            return []
        gx, gy = best
    open_set = [(0, sx, sy)]
    came_from = {}
    g_score = {(sx, sy): 0}
    max_nodes = min(cols * rows, 4000)  # Cap search space
    nodes_expanded = 0
    while open_set and nodes_expanded < max_nodes:
        _, cx, cy = heapq.heappop(open_set)
        nodes_expanded += 1
        if (cx, cy) == (gx, gy):
            path = []
            while (cx, cy) in came_from:
                path.append((cx, cy))
                cx, cy = came_from[(cx, cy)]
            path.reverse()
            return path
        for dx, dy in (
            (1, 0), (-1, 0), (0, 1), (0, -1),
            (1, 1), (-1, 1), (1, -1), (-1, -1),
        ):
            nx, ny = cx + dx, cy + dy
            if 0 <= nx < cols and 0 <= ny < rows and grid[ny][nx]:
                if dx != 0 and dy != 0:
                    if not grid[cy][nx] or not grid[ny][cx]:
                        continue
                step_cost = _SQRT2 if (dx != 0 and dy != 0) else 1.0
                tentative = g_score[(cx, cy)] + step_cost
                key = (nx, ny)
                if tentative < g_score.get(key, float("inf")):
                    came_from[key] = (cx, cy)
                    g_score[key] = tentative
                    f = tentative + _heuristic(nx, ny, gx, gy)
                    heapq.heappush(open_set, (f, nx, ny))
    return []
_next_enemy_id = 0
class Enemy:
    def __init__(self, x, y):
        global _next_enemy_id
        self.uid = _next_enemy_id
        _next_enemy_id += 1
        self.rect = pygame.Rect(x, y, 24, 24)
        self.speed = 2
        self.angle = 0
        self.hp = 100
        self.state = "IDLE"
        self.state_timer = 0
        self.fire_cooldown = 0
        self.lost_sight_timer = 0
        self.idle_angle = random.uniform(0, math.pi * 2)
        self.idle_switch_timer = random.randint(60, 180)
        self.search_timer = 0
        self.alerted = False
        self.drops_mag = False
        self.path = []
        self.path_timer = 0
        self.path_waypoint_timer = 0
        self.patrol_points = []
        self.patrol_idx = 0
        self.distraction_target = None
        self.last_seen_pos = None
        self.combat_alert_timer = 0
        self.sweep_points = []
        # Stuck detection
        self._last_pos = (x, y)
        self._stuck_timer = 0
        self.fire_react_cooldown = 0
        self.fire_discipline = random.uniform(0.65, 1.0)
        self.flinch_timer = 0
        self.entry_grace = 0
        self.breach_entry_target = None
        self.breach_entering = False
        self.breach_entry_source = ""
        self._prev_state = "IDLE"
    def _can_pass(self, target_x, target_y, walls):
        cx, cy = self.rect.centerx, self.rect.centery
        dx = target_x - cx
        dy = target_y - cy
        dist = math.hypot(dx, dy)
        if dist == 0:
            return True
        radius = 10
        nx = -dy / dist * radius
        ny = dx / dist * radius
        for wall in walls:
            if wall.clipline((cx, cy), (target_x, target_y)):
                return False
            if wall.clipline((cx + nx, cy + ny), (target_x + nx, target_y + ny)):
                return False
            if wall.clipline((cx - nx, cy - ny), (target_x - nx, target_y - ny)):
                return False
        return True
    def _smooth_path(self, walls):
        if not self.path:
            return
        for i in range(min(len(self.path) - 1, 5), 0, -1):
            col, row = self.path[i]
            wx = col * TILE_SIZE + TILE_SIZE // 2
            wy = row * TILE_SIZE + TILE_SIZE // 2
            if self._can_pass(wx, wy, walls):
                self.path = self.path[i:]
                break
    def _build_sweep_points(self, cx, cy, walls):
        grid, map_cols, map_rows = _build_walk_grid(walls)
        if not grid:
            return []
        offsets = [
            (TILE_SIZE * 3, 0), (-TILE_SIZE * 3, 0),
            (0, TILE_SIZE * 3), (0, -TILE_SIZE * 3),
            (TILE_SIZE, TILE_SIZE), (-TILE_SIZE, TILE_SIZE),
            (TILE_SIZE, -TILE_SIZE), (-TILE_SIZE, -TILE_SIZE),
            (TILE_SIZE * 5, 0), (-TILE_SIZE * 5, 0),
            (0, TILE_SIZE * 5), (0, -TILE_SIZE * 5),
        ]
        random.shuffle(offsets)
        search_offsets = []
        for dr in range(-2, 3):
            for dc in range(-2, 3):
                search_offsets.append((dr, dc))
        search_offsets.sort(key=lambda p: abs(p[0]) + abs(p[1]))
        points = []
        for dx, dy in offsets:
            wx, wy = cx + dx, cy + dy
            col = max(0, min(int(wx // TILE_SIZE), map_cols - 1))
            row = max(0, min(int(wy // TILE_SIZE), map_rows - 1))
            found = False
            for dr, dc in search_offsets:
                nr, nc = row + dr, col + dc
                if 0 <= nc < map_cols and 0 <= nr < map_rows and grid[nr][nc]:
                    col, row = nc, nr
                    found = True
                    break
            if not found:
                continue
            tx = col * TILE_SIZE + TILE_SIZE // 2
            ty = row * TILE_SIZE + TILE_SIZE // 2
            points.append((tx, ty))
            if len(points) >= 4:
                break
        return points
    def update(self, px, py, walls, bullets, particles, enemies=None, fire_zones=None):
        # Delta-time multiplier: speed values in this module are in px/frame,
        # so multiply by (60 * dt) to get px this frame at any FPS.
        # At 60 FPS dt=1/60, so 60*dt = 1 — identical to old behavior.
        _dt_mul = 60.0 * State.dt
        self._dt_mul = _dt_mul  # cached for _apply_move
        self.state_timer += 1
        self.fire_cooldown -= 1
        if self.entry_grace > 0:
            self.entry_grace -= 1
        if self.fire_react_cooldown > 0:
            self.fire_react_cooldown -= 1
        if self.combat_alert_timer > 0:
            self.combat_alert_timer -= 1
        # — Flinch on state change to chase/attack —
        if self.state != self._prev_state:
            if self.state in ("CHASE", "ATTACK") and self._prev_state in ("IDLE", "PATROL"):
                self.flinch_timer = 8
        self._prev_state = self.state
        if self.flinch_timer > 0:
            self.flinch_timer -= 1
        can_see = self._can_see(px, py, walls)
        if can_see:
            self.last_seen_pos = (px, py)
            self.lost_sight_timer = 0
            self.combat_alert_timer = 600
            if self.alerted and self.state not in ("CHASE", "ATTACK"):
                self.state = "CHASE"
        else:
            self.lost_sight_timer += 1
        # Sight cone detection
        cone_half = math.radians(60) if self.alerted else math.radians(45)
        dist_to_player = math.hypot(px - self.rect.centerx, py - self.rect.centery)
        detect_range = ENEMY_DETECT_RANGE * (1.5 if self.alerted else 1)
        if not self.alerted and dist_to_player < detect_range:
            in_sight_cone = False
            if dist_to_player < 60 and can_see:
                # Close-range peripheral awareness. Stops enemies being blind when
                # the player walks almost beside them.
                in_sight_cone = True
            else:
                angle_to_player = math.atan2(
                    py - self.rect.centery, px - self.rect.centerx
                )
                diff = abs(angle_to_player - self.angle)
                if diff > math.pi:
                    diff = 2 * math.pi - diff
                if diff < cone_half and can_see:
                    in_sight_cone = True
            if in_sight_cone:
                self.alerted = True
                self.state = "CHASE"
                self.last_seen_pos = (px, py)
                self.combat_alert_timer = 600
        # Soft repulsion — capped to prevent wall-pushing
        repulse_x, repulse_y = 0.0, 0.0
        personal_space = 40.0
        if enemies:
            for other in enemies:
                if other is not self and other.state in ("CHASE", "ATTACK"):
                    dx = self.rect.centerx - other.rect.centerx
                    dy = self.rect.centery - other.rect.centery
                    dist = math.hypot(dx, dy)
                    if 0 < dist < personal_space:
                        force = (personal_space - dist) / personal_space
                        repulse_x += (dx / dist) * force * 0.6
                        repulse_y += (dy / dist) * force * 0.6
            # Cap repulsion magnitude
            rep_mag = math.hypot(repulse_x, repulse_y)
            if rep_mag > self.speed * 0.5:
                scale = (self.speed * 0.5) / rep_mag
                repulse_x *= scale
                repulse_y *= scale
        # — FIRE ZONE AWARENESS —
        # Only care about fire that's actually close.
        nearest_fire = None
        nearest_fire_dist = float("inf")
        if fire_zones:
            for fz in fire_zones:
                fd = math.hypot(self.rect.centerx - fz.x, self.rect.centery - fz.y)
                if fd < fz.radius + 40 and fd < nearest_fire_dist:
                    nearest_fire = fz
                    nearest_fire_dist = fd
        if nearest_fire:
            in_core = nearest_fire_dist < nearest_fire.radius * 0.45
            near_edge = nearest_fire_dist < nearest_fire.radius * 0.75
            player_visible_and_shootable = (
                can_see and dist_to_player <= ENEMY_ATTACK_RANGE * 1.1
            )
            if in_core:
                if self.fire_react_cooldown <= 0:
                    self.fire_react_cooldown = random.randint(20, 36)
                else:
                    if random.random() < 0.55:
                        move_x = repulse_x * 0.25
                        move_y = repulse_y * 0.25
                        self._apply_move(move_x, move_y, walls)
                        return
                self.alerted = True
                self.combat_alert_timer = max(self.combat_alert_timer, 480)
                self.path = []
                self.distraction_target = None
                flee_angle = math.atan2(
                    self.rect.centery - nearest_fire.y,
                    self.rect.centerx - nearest_fire.x,
                )
                flee_angle += random.uniform(-0.45, 0.45)
                flee_speed = self.speed * random.uniform(1.05, 1.25) * self.fire_discipline
                move_x = math.cos(flee_angle) * flee_speed + repulse_x * 0.3
                move_y = math.sin(flee_angle) * flee_speed + repulse_y * 0.3
                self.angle = flee_angle
                self._apply_move(move_x, move_y, walls)
                if can_see:
                    self.last_seen_pos = (px, py)
                    self.state = "CHASE"
                return
            elif near_edge and not player_visible_and_shootable:
                avoid_angle = math.atan2(
                    self.rect.centery - nearest_fire.y,
                    self.rect.centerx - nearest_fire.x,
                )
                repulse_x += math.cos(avoid_angle) * 0.7 * self.fire_discipline
                repulse_y += math.sin(avoid_angle) * 0.7 * self.fire_discipline
        if self.state == "IDLE":
            self.idle_switch_timer -= 1
            if self.idle_switch_timer <= 0:
                self.idle_angle = random.uniform(0, math.pi * 2)
                self.idle_switch_timer = random.randint(60, 180)
            self._ease_toward_angle(self.idle_angle, 0.035)
        elif self.state == "PATROL":
            if not self.patrol_points:
                self.state = "IDLE"
            else:
                target_x, target_y = self.patrol_points[self.patrol_idx]
                dx = target_x - self.rect.centerx
                dy = target_y - self.rect.centery
                dist = math.hypot(dx, dy)
                if dist < 10:
                    self.patrol_idx = (self.patrol_idx + 1) % len(self.patrol_points)
                else:
                    self._ease_toward_angle(math.atan2(dy, dx), 0.08)
                    move_x = math.cos(self.angle) * self.speed
                    move_y = math.sin(self.angle) * self.speed
                    self.rect.x += move_x * _dt_mul
                    for wall in walls:
                        if self.rect.colliderect(wall):
                            if move_x > 0: self.rect.right = wall.left
                            elif move_x < 0: self.rect.left = wall.right
                    self.rect.y += move_y * _dt_mul
                    for wall in walls:
                        if self.rect.colliderect(wall):
                            if move_y > 0: self.rect.bottom = wall.top
                            elif move_y < 0: self.rect.top = wall.bottom
        elif self.state == "SEARCH":
            if getattr(self, "distraction_target", None):
                tx, ty = self.distraction_target
                self.angle = math.atan2(ty - self.rect.centery, tx - self.rect.centerx)
                dist_to_distraction = math.hypot(tx - self.rect.centerx, ty - self.rect.centery)
                if dist_to_distraction > 20:
                    self._update_pathfinding(tx, ty, walls)
                    self._move_toward(tx, ty, walls, repulse_x, repulse_y)
                else:
                    self.distraction_target = None
                    self.path = []
                    if self.sweep_points:
                        self.distraction_target = self.sweep_points.pop(0)
            else:
                # Scan — face toward last seen position
                if self.last_seen_pos:
                    target_a = math.atan2(
                        self.last_seen_pos[1] - self.rect.centery,
                        self.last_seen_pos[0] - self.rect.centerx,
                    )
                    sweep_offset = math.sin(pygame.time.get_ticks() * 0.003) * math.radians(60)
                    target_a += sweep_offset
                    diff = (target_a - self.angle + math.pi) % (math.pi * 2) - math.pi
                    self.angle += diff * 0.05
                else:
                    self.idle_switch_timer -= 1
                    if self.idle_switch_timer <= 0:
                        self.idle_angle = random.uniform(0, math.pi * 2)
                        self.idle_switch_timer = random.randint(30, 90)
                    self.angle += (self.idle_angle - self.angle) * 0.05
            self.search_timer -= 1
            if self.search_timer <= 0:
                # FIX: Only de-alert if combat_alert_timer is truly expired
                if self.combat_alert_timer <= 0:
                    self.alerted = False
                    self.state = "PATROL" if self.patrol_points else "IDLE"
                    self.patrol_idx = 0
                    self.distraction_target = None
                    self.sweep_points = []
                    self.path = []
                else:
                    # Still combat-aware — do one more search sweep
                    self.search_timer = 120
                    self.distraction_target = None
                    self.sweep_points = []
                    self.path = []
        elif self.state == "CHASE":
            if self.last_seen_pos:
                tx, ty = self.last_seen_pos
                self._update_pathfinding(tx, ty, walls)
                dist_to_last = math.hypot(tx - self.rect.centerx, ty - self.rect.centery)
                if can_see and dist_to_player <= ENEMY_ATTACK_RANGE:
                    self.state = "ATTACK"
                    self.path = []
                elif dist_to_last < 20 and not can_see and self.lost_sight_timer > 60:
                    # Reached last known position, can't see — search
                    self.state = "SEARCH"
                    self.search_timer = 300 if self.combat_alert_timer > 0 else 180
                    self.sweep_points = self._build_sweep_points(tx, ty, walls)
                    if self.sweep_points:
                        self.distraction_target = self.sweep_points.pop(0)
                    self.path = []
                else:
                    self._move_toward(tx, ty, walls, repulse_x, repulse_y)
            else:
                self.state = "SEARCH"
                self.search_timer = 180
                self.distraction_target = None
                self.path = []
        elif self.state == "ATTACK":
            if can_see or self.lost_sight_timer < 45:
                self.alerted = True
                self.combat_alert_timer = 600
                # Always face player — with easing for human delay
                target_a = math.atan2(py - self.rect.centery, px - self.rect.centerx)
                if self.flinch_timer > 0:
                    flinch_jitter = math.sin(pygame.time.get_ticks() * 0.4) * 0.15
                    target_a += flinch_jitter
                self._ease_toward_angle(target_a, 0.18)
                optimal_min = ENEMY_ATTACK_RANGE * 0.35
                optimal_max = ENEMY_ATTACK_RANGE * 0.85
                # Use a larger threshold before going back to CHASE
                out_of_range = dist_to_player > ENEMY_ATTACK_RANGE * 1.2
                if out_of_range:
                    self.state = "CHASE"
                else:
                    # Shoot when ready
                    if can_see and self.fire_cooldown <= 0 and self.entry_grace <= 0:
                        self._shoot(px, py, bullets, particles, walls)
                        # Ambience-wave enemies (post-faction cleanup team) fire
                        # faster than the global default — 30 frames vs 48.
                        # This is a per-enemy override so it doesn't affect
                        # Mission 1 or the sniper phase.
                        self.fire_cooldown = 30 if getattr(self, "is_ambience_wave", False) else ENEMY_FIRE_COOLDOWN
                    too_close = dist_to_player < optimal_min
                    too_far = dist_to_player > optimal_max
                    if too_close:
                        away_angle = math.atan2(
                            self.rect.centery - py, self.rect.centerx - px
                        )
                        move_x = math.cos(away_angle) * self.speed * 0.6 + repulse_x
                        move_y = math.sin(away_angle) * self.speed * 0.6 + repulse_y
                        self._apply_move(move_x, move_y, walls)
                    elif too_far:
                        self._update_pathfinding(px, py, walls)
                        self._move_toward(px, py, walls, repulse_x, repulse_y)
                    elif not can_see:
                        # Strafe to regain LOS
                        strafe_angle = self.angle + math.pi / 2
                        if (pygame.time.get_ticks() // 1000) % 2 == 0:
                            strafe_angle = self.angle - math.pi / 2
                        move_x = math.cos(strafe_angle) * self.speed * 0.6 + repulse_x
                        move_y = math.sin(strafe_angle) * self.speed * 0.6 + repulse_y
                        self._apply_move(move_x, move_y, walls)
                    else:
                        # In optimal range with LOS — just apply repulsion
                        if math.hypot(repulse_x, repulse_y) > 0.1:
                            self._apply_move(repulse_x, repulse_y, walls)
            else:
                # Lost sight for too long
                if self.last_seen_pos:
                    self.state = "CHASE"
                else:
                    self.state = "SEARCH"
                    self.search_timer = 180
        # Hard overlap resolution
        if enemies:
            for other in enemies:
                if other is not self:
                    if self.rect.colliderect(other.rect):
                        dx = self.rect.centerx - other.rect.centerx
                        dy = self.rect.centery - other.rect.centery
                        dist = math.hypot(dx, dy)
                        if dist == 0:
                            dx = random.choice([-1, 1])
                            dy = random.choice([-1, 1])
                            dist = math.hypot(dx, dy)
                        overlap = (self.rect.width / 2 + other.rect.width / 2) - dist
                        if overlap > 0:
                            push_force = min(overlap / 2, 3.0)
                            push_x = (dx / dist) * push_force
                            push_y = (dy / dist) * push_force
                            self.rect.x += push_x * _dt_mul
                            for wall in walls:
                                if self.rect.colliderect(wall):
                                    if push_x > 0: self.rect.right = wall.left
                                    elif push_x < 0: self.rect.left = wall.right
                            self.rect.y += push_y * _dt_mul
                            for wall in walls:
                                if self.rect.colliderect(wall):
                                    if push_y > 0: self.rect.bottom = wall.top
                                    elif push_y < 0: self.rect.top = wall.bottom
    def _ease_toward_angle(self, target_angle, speed=0.08):
        diff = (target_angle - self.angle + math.pi) % (math.pi * 2) - math.pi
        if abs(diff) < 0.01:
            self.angle = target_angle
        else:
            self.angle += diff * speed
            self.angle = (self.angle + math.pi) % (math.pi * 2) - math.pi
    def _can_see(self, px, py, walls, from_pos=None):
        if from_pos is None:
            from_pos = (self.rect.centerx, self.rect.centery)
        dist = math.hypot(px - from_pos[0], py - from_pos[1])
        # Alerted enemies should not suddenly become blind at medium range.
        aware = self.alerted or self.combat_alert_timer > 0
        sight_range = ENEMY_DETECT_RANGE * (1.8 if aware else 1.35)
        if dist > sight_range:
            return False
        # Check several points around the player instead of only the exact center.
        # This prevents tiny wall/corner chips from making AI instantly "forget" you.
        target_points = [
            (px, py),
            (px - 10, py),
            (px + 10, py),
            (px, py - 10),
            (px, py + 10),
            (px - 8, py - 8),
            (px + 8, py - 8),
            (px - 8, py + 8),
            (px + 8, py + 8),
        ]
        for tx, ty in target_points:
            blocked = False
            for wall in walls:
                if wall.clipline(from_pos, (tx, ty)):
                    blocked = True
                    break
            if not blocked:
                return True
        return False
    def _update_pathfinding(self, px, py, walls):
        self.path_timer -= 1
        if self.path_timer > 0:
            return
        dist = math.hypot(px - self.rect.centerx, py - self.rect.centery)
        if dist < ENEMY_ATTACK_RANGE * 1.5:
            self.path_timer = max(5, PATH_RECALC_INTERVAL // 3)
        else:
            self.path_timer = PATH_RECALC_INTERVAL
        grid, map_cols, map_rows = _build_walk_grid(walls)
        if map_cols == 0 or map_rows == 0:
            self.path = []
            return
        start_col = int(self.rect.centerx // TILE_SIZE)
        start_row = int(self.rect.centery // TILE_SIZE)
        goal_col = int(px // TILE_SIZE)
        goal_row = int(py // TILE_SIZE)
        if start_col == goal_col and start_row == goal_row:
            self.path = []
            return
        self.path = _astar((start_col, start_row), (goal_col, goal_row), grid)
```

```
                    self._smooth_path(walls)
    def _move_toward(self, px, py, walls, impulse_x=0, impulse_y=0):
        if self.path:
            col, row = self.path[0]
            target_x = col * TILE_SIZE + TILE_SIZE // 2
            target_y = row * TILE_SIZE + TILE_SIZE // 2
            dx = target_x - self.rect.centerx
            dy = target_y - self.rect.centery
            dist_sq = dx * dx + dy * dy
            reach = PATH_WAYPOINT_REACH * 1.25
            if dist_sq < reach * reach:
                self.path.pop(0)
                self._smooth_path(walls)
                self.path_waypoint_timer = 0
                if self.path:
                    col, row = self.path[0]
                    target_x = col * TILE_SIZE + TILE_SIZE // 2
                    target_y = row * TILE_SIZE + TILE_SIZE // 2
                else:
                    target_x, target_y = px, py
            else:
                # FIX: Stuck detection based on actual position change
                cx, cy = self.rect.centerx, self.rect.centery
                moved = math.hypot(cx - self._last_pos[0], cy - self._last_pos[1])
                if moved < 0.5:
                    self._stuck_timer += 1
                else:
                    self._stuck_timer = 0
                self._last_pos = (cx, cy)
                if self._stuck_timer > 30:
                    self._stuck_timer = 0
                    if self.path:
                        self.path.pop(0)
                    self._smooth_path(walls)
                    self.path_timer = 0  # force repath next frame
            self._ease_toward_angle(
                math.atan2(target_y - self.rect.centery, target_x - self.rect.centerx), 0.07
            )
            move_x = math.cos(self.angle) * self.speed + impulse_x
            move_y = math.sin(self.angle) * self.speed + impulse_y
            slide_tx, slide_ty = target_x, target_y
        else:
            # No path - use direct steering
            desired_angle = math.atan2(py - self.rect.centery, px - self.rect.centerx)
            num_dirs = 16
            probe_len = PROBE_LEN
            best_angle = desired_angle
            best_score = -999.0
            for i in range(num_dirs):
                angle_offset = -math.pi / 2 + (i / (num_dirs - 1)) * math.pi
                candidate = desired_angle + angle_offset
                alignment = math.cos(candidate - desired_angle)
                probe_x = self.rect.centerx + math.cos(candidate) * probe_len
                probe_y = self.rect.centery + math.sin(candidate) * probe_len
                probe_rect = pygame.Rect(probe_x - 10, probe_y - 10, 20, 20)
                blocked = any(w.colliderect(probe_rect) for w in walls)
                short_probe_x = self.rect.centerx + math.cos(candidate) * (probe_len * 0.4)
                short_probe_y = self.rect.centery + math.sin(candidate) * (probe_len * 0.4)
                short_probe_rect = pygame.Rect(short_probe_x - 8, short_probe_y - 8, 16, 16)
                short_blocked = any(w.colliderect(short_probe_rect) for w in walls)
                score = alignment - (5.0 if blocked else 0.0) - (3.0 if short_blocked else 0.0)
                if score > best_score:
                    best_score = score
                    best_angle = candidate
            diff = (best_angle - self.angle + math.pi) % (math.pi * 2) - math.pi
            self.angle += diff * 0.05
            move_x = math.cos(self.angle) * self.speed + impulse_x
            move_y = math.sin(self.angle) * self.speed + impulse_y
            slide_tx, slide_ty = px, py
        self.rect.x += move_x * self._dt_mul
        blocked_x = False
        for wall in walls:
            if self.rect.colliderect(wall):
                if move_x > 0:
                    self.rect.right = wall.left
                elif move_x < 0:
                    self.rect.left = wall.right
                blocked_x = True
        self.rect.y += move_y * self._dt_mul
        blocked_y = False
        for wall in walls:
            if self.rect.colliderect(wall):
                if move_y > 0:
                    self.rect.bottom = wall.top
                elif move_y < 0:
                    self.rect.top = wall.bottom
                blocked_y = True
        if blocked_x and not blocked_y:
            if abs(slide_ty - self.rect.centery) > 4:
                slide_dy = self.speed if slide_ty > self.rect.centery else -self.speed
                self.rect.y += slide_dy * self._dt_mul
                for wall in walls:
                    if self.rect.colliderect(wall):
                        if slide_dy > 0: self.rect.bottom = wall.top
                        elif slide_dy < 0: self.rect.top = wall.bottom
        elif blocked_y and not blocked_x:
            if abs(slide_tx - self.rect.centerx) > 4:
                slide_dx = self.speed if slide_tx > self.rect.centerx else -self.speed
                self.rect.x += slide_dx * self._dt_mul
                for wall in walls:
                    if self.rect.colliderect(wall):
                        if slide_dx > 0: self.rect.right = wall.left
                        elif slide_dx < 0: self.rect.left = wall.right
        elif blocked_x and blocked_y:
            # Both axes blocked - force repath
            self.path = []
            self.path_timer = 0
    def _apply_move(self, move_x, move_y, walls):
        # Delta-time aware: callers pass per-frame movement; multiply by _dt_mul.
        _dt_mul = getattr(self, "_dt_mul", 1.0)
        move_x *= _dt_mul
        move_y *= _dt_mul
        self.rect.x += move_x
        for wall in walls:
            if self.rect.colliderect(wall):
                if move_x > 0: self.rect.right = wall.left
                elif move_x < 0: self.rect.left = wall.right
        self.rect.y += move_y
        for wall in walls:
            if self.rect.colliderect(wall):
                if move_y > 0: self.rect.bottom = wall.top
                elif move_y < 0: self.rect.top = wall.bottom
    def _gun_pos(self):
        """World-space enemy muzzle position.
        Matches Enemy.draw():
        arm_len = 15
        grip offset = 4
        gun length = 20
        """
        reach = 15 + 4 + 20
        return (
            self.rect.centerx + math.cos(self.angle) * reach,
            self.rect.centery + math.sin(self.angle) * reach,
        )
    def _shoot(self, px, py, bullets, particles, walls):
        gx, gy = self._gun_pos()
        spread = random.uniform(-0.06, 0.06)
        # Spawn exactly at the muzzle, not 5 pixels ahead of it.
        bx = gx
        by = gy
        damage = 8 if getattr(self, "is_ambush_enemy", False) else ENEMY_DAMAGE
        bullets.append({
            "x": bx, "y": by,
            "dx": math.cos(self.angle + spread) * 10,
            "dy": math.sin(self.angle + spread) * 10,
            "life": 60,
            "owner": "enemy",
            "damage": damage,
        })
        for _ in range(4):
            particles.append(Particle(bx, by, (255, 200, 100)))
        audio.play_3d(
            "gunshot_distant",
            (self.rect.centerx, self.rect.centery),
            (px, py),
            base_volume=0.6,
        )
    def hear_shot(self, sx: float, sy: float, walls: list) -> None:
        dist = math.hypot(sx - self.rect.centerx, sy - self.rect.centery)
        if dist >= ENEMY_HEAR_RADIUS:
            return
        has_los = self._has_los(sx, sy, walls)
        if not has_los:
            return
        noise = random.randint(-TILE_SIZE // 4, TILE_SIZE // 4)
        noisy_pos = (sx + noise, sy + noise)
        if self.state in ("CHASE", "ATTACK"):
            if dist < TILE_SIZE * 4:
                self.last_seen_pos = noisy_pos
                self.combat_alert_timer = max(self.combat_alert_timer, 360)
            return
        if self.state == "SEARCH":
            self.combat_alert_timer = max(self.combat_alert_timer, 240)
            self.sweep_points = [noisy_pos]
            self.distraction_target = self.sweep_points.pop(0)
            self.search_timer = max(self.search_timer, 180)
            return
        self.alerted = True
        self.state = "SEARCH"
        self.search_timer = 240
        self.combat_alert_timer = 300
        self.distraction_target = noisy_pos
        self.sweep_points = []
        self.path = []
        self.path_timer = 0
    def on_hit(self, shooter_x: float, shooter_y: float) -> None:
        self.alerted = True
        self.combat_alert_timer = 480
        noise = random.randint(-TILE_SIZE // 2, TILE_SIZE // 2)
        self.last_seen_pos = (shooter_x + noise, shooter_y + noise)
        if self.state not in ("ATTACK",):
            self.state = "CHASE"
        self.sweep_points = []
        self.path = []
        self.path_timer = 0
        if self.fire_cooldown <= 0:
            self.fire_cooldown = random.randint(8, 20)
    def draw(self, surface, offset_x, offset_y):
        draw_x = self.rect.centerx - offset_x
        draw_y = self.rect.centery - offset_y
        a = self.angle
        d_x = math.cos(a)
        d_y = math.sin(a)
        perp_x = math.cos(a + math.pi / 2)
        perp_y = math.sin(a + math.pi / 2)
        surface.blit(_enemy_shadow, (int(draw_x - 17), int(draw_y - 12)))
        is_swat = getattr(self, "is_commander", False)
        is_amb = getattr(self, "is_ambush_enemy", False)
        if is_swat:
            uniform_col = (15, 30, 45)
            skin_col = (175, 145, 120)
        elif is_amb:
            uniform_col = C_RED_UNIFORM
            skin_col = C_ENEMY_SKIN
        else:
            uniform_col = C_ENE_UNIFORM
            skin_col = C_ENEMY_SKIN
        body_w, body_h = 28, 14
        cos_b, sin_b = math.cos(a), math.sin(a)
        torso_pts = []
        for bx, by in [
            (-body_w // 2, -body_h // 2),
            (body_w // 2, -body_h // 2),
            (body_w // 2, body_h // 2),
            (-body_w // 2, body_h // 2),
        ]:
            rx = bx * cos_b - by * sin_b
            ry = bx * sin_b + by * cos_b
            torso_pts.append((draw_x + rx, draw_y + ry))
        pygame.draw.polygon(surface, uniform_col, torso_pts)
        pygame.draw.line(
            surface,
            (uniform_col[0] + 30, uniform_col[1] + 8, uniform_col[2] + 4),
            (draw_x - d_x * 8, draw_y - d_y * 8),
            (draw_x + d_x * 8, draw_y + d_y * 8),
            4,
        )
        lh_x = draw_x + perp_x * 10
        lh_y = draw_y + perp_y * 10
        rh_x = draw_x - perp_x * 10
        rh_y = draw_y - perp_y * 10
        arm_len = 15
        hand_x = draw_x + d_x * arm_len
        hand_y = draw_y + d_y * arm_len
        pygame.draw.line(
            surface, uniform_col, (lh_x, lh_y),
            (hand_x - perp_x * 3, hand_y - perp_y * 3), 5
        )
        pygame.draw.line(
            surface, uniform_col, (rh_x, rh_y),
            (hand_x + perp_x * 3, hand_y + perp_y * 3), 5
        )
        grip_x = draw_x + d_x * (arm_len + 4)
        grip_y = draw_y + d_y * (arm_len + 4)
        pygame.draw.circle(surface, skin_col, (int(grip_x), int(grip_y)), 4)
        gun_start_x = grip_x - d_x * 4
        gun_start_y = grip_y - d_y * 4
        gun_end_x = grip_x + d_x * 20
        gun_end_y = grip_y + d_y * 20
        pygame.draw.line(
            surface, (35, 35, 45), (gun_start_x, gun_start_y), (gun_end_x, gun_end_y), 5
        )
        pygame.draw.circle(surface, (25, 45, 30), (int(gun_end_x), int(gun_end_y)), 3)
        head_x = draw_x + d_x * 4
        head_y = draw_y + d_y * 4
        head_r = 8
        pygame.draw.circle(surface, skin_col, (int(head_x), int(head_y)), head_r)

        # Draw armbands on standard ENE guards
        if not is_swat and not is_amb:
            band_x1 = draw_x + perp_x * 9
            band_y1 = draw_y + perp_y * 9
            band_x2 = band_x1 + d_x * 4
            band_y2 = band_y1 + d_y * 4
            pygame.draw.line(surface, C_RED_ARMBAND,
                             (int(band_x1), int(band_y1)),
                             (int(band_x2), int(band_y2)), 3)

        if is_swat:
            helm_x = int(head_x - d_x * 2)
            helm_y = int(head_y - d_y * 2)
            pygame.draw.circle(surface, (30, 30, 35), (helm_x, helm_y), head_r + 1)
            pygame.draw.circle(surface, skin_col, (int(head_x), int(head_y)), head_r)
            pip1_x = draw_x + perp_x * 8 - d_x * 3
            pip1_y = draw_y + perp_y * 8 - d_y * 3
            pip2_x = pip1_x - d_x * 4
            pip2_y = pip1_y - d_y * 4
            pygame.draw.circle(surface, (220, 200, 60), (int(pip1_x), int(pip1_y)), 2)
            pygame.draw.circle(surface, (220, 200, 60), (int(pip2_x), int(pip2_y)), 2)
            band_x1 = draw_x + perp_x * 10
            band_y1 = draw_y + perp_y * 10
            band_x2 = band_x1 + d_x * 6
            band_y2 = band_y1 + d_y * 6
            pygame.draw.line(surface, (200, 50, 50),
                             (int(band_x1), int(band_y1)),
                             (int(band_x2), int(band_y2)), 3)
            label = font_terminal_small.render("SWAT", True, (240, 240, 60))
            surface.blit(label, (int(draw_x - label.get_width() // 2), int(draw_y - 30)))
        elif is_amb:
            # Draw red bandana on the head
            pygame.draw.circle(surface, C_RED_BANDANA, (int(head_x - d_x * 2), int(head_y - d_y * 2)), head_r - 1)
            tail1_x = int(head_x - d_x * 8 + perp_x * 3)
            tail1_y = int(head_y - d_y * 8 + perp_y * 3)
            tail2_x = int(head_x - d_x * 8 - perp_x * 3)
            tail2_y = int(head_y - d_y * 8 - perp_y * 3)
            pygame.draw.line(surface, C_RED_BANDANA, (int(head_x - d_x * 4), int(head_y - d_y * 4)), (tail1_x, tail1_y), 3)
            pygame.draw.line(surface, C_RED_BANDANA, (int(head_x - d_x * 4), int(head_y - d_y * 4)), (tail2_x, tail2_y), 3)

            label = font_terminal_small.render("RED", True, (255, 100, 100))
            surface.blit(label, (int(draw_x - label.get_width() // 2), int(draw_y - 30)))
        else:
            pygame.draw.arc(
                surface, (35, 30, 30),
                (int(head_x - head_r), int(head_y - head_r), head_r * 2, head_r * 2),
                a + 1.4, a + math.pi * 2 - 1.4, 3
            )
        face_x = head_x + d_x * 4
        face_y = head_y + d_y * 4
        for eye_off in (-0.45, 0.45):
            ex = face_x + math.cos(a + eye_off) * 3
            ey = face_y + math.sin(a + eye_off) * 3
            pygame.draw.circle(surface, (30, 25, 20), (int(ex), int(ey)), 1)
def enemy_hear_shot(sx, sy, enemies, walls):
    for e in enemies:
        e.hear_shot(sx, sy, walls)
```